

# CPSC 335 - Algorithm Engineering

## Project 1

Group Members:

Vibhor Bhargava: [vibhor.b@csu.fullerton.edu](mailto:vibhor.b@csu.fullerton.edu)

Aly Sayed: [alysayed@csu.fullerton.edu](mailto:alysayed@csu.fullerton.edu)

Matthew Aarom Gabriel: [Matthew.Gabriel@csu.fullerton.edu](mailto:Matthew.Gabriel@csu.fullerton.edu)

# Algorithm 1 - Greedy Approach to Hamiltonian Problem

## Description:

The purpose of Algorithm 1 is to find the preferred starting city that allows the car to travel around all of the cities without running out of fuel. Our algorithm uses a greedy approach. It starts at the first city and keeps track of the current amount of travel distance available. At each city, the fuel available is multiplied by the MPG and added to the current amount. The distance to the next city is then subtracted. If the value becomes negative, that city cannot be the preferred starting city, so the algorithm moves the starting position to the next city and resets the current amount to zero.

## Efficiency Analysis:

We used the step count method to analyze this algorithm. Let  $n$  represent the number of cities. The algorithm contains one loop that goes through the cities from the beginning to the end. Each iteration performs a constant number of operations, such as addition, multiplication, subtraction, comparison, and assignment.

Therefore, the running time can be represented as:

$$T(n) = cn + c_0$$

Where  $c$  represents the constant amount of work performed during each iteration.

After removing constants and lower - order terms:

$$O(n)$$

Therefore, Algorithm 1 has a linear time complexity of  $O(n)$ .

## Algorithm 2 - Connecting Pairs of Persons

### Description:

The purpose of Algorithm 2 is to determine the minimum number of swaps needed so that every couple is sitting next to each other. The algorithm goes through the row two people at a time. It checks whether the two people currently next to each other form a couple. If they do not, it determines the ID of the correct partner and searches through the row for that person. The correct partner is then swapped into the position next to their partner, and the number of swaps is increased.

### Efficiency Analysis:

We again used the step count method. Let  $n$  represent the number of people in the row. The outer loop goes through the row, and when a pair is incorrect, another loop searches through the row for the correct partner.

In the worst case, the outer loop performs approximately  $n/2$  iterations, and the inner search can perform up to  $n$  iterations:

$$T(n) \approx \frac{n}{2} (n)$$

$$T(n) \approx \frac{n^2}{2}$$

Ignoring the constant:  $O(n^2)$

Therefore, Algorithm 2 has a quadratic worst-case time complexity of  $O(n^2)$  based on the approach shown in the pseudocode / current code.

## Conclusion:

For project 1, we implemented two different algorithmic solutions. Algorithm 1 uses a greedy approach and has a time complexity of  $O(n)$ . . Algorithm 2 searches for the correct partner when a couple is not seated together and has a worst-case time complexity of  $O(n^2)$ . These two algorithms were implemented in Python, and separate README files provide instructions for executing the programs.

The assignment requires the mathematical analysis and efficiency class for both algorithms, so those two analysis sections cover that requirement.